arXiv:2608.22237v2 [cs.AI] 7 Nov 2026

Read Less, Solve More: Token-Efficient Sparse Reading of External Content for AI Agents

Zedong Liu[1,2], **Jiaan Wu**[1,2], **Xinyang Ma**[1,2], **Le Xu**[1,2],
Kai Wang[3], **Yuanchao Hu**[3], **Dingwen Tao**[1,2], **Guangming Tan**[1,2]

[1]University of Chinese Academy of Sciences, Beijing, China
[2]Institute of Computing Technology, Chinese Academy of Sciences, Beijing, China
[3]Songshan Lake Materials Laboratory, Dongguan, China

Abstract

Web-facing AI agents repeatedly access webpages, documents, repositories, and tool responses, yet reading interfaces often expose entire objects when only sparse evidence is needed. This over-reading increases token and latency costs and can dilute relevant evidence. We present SparseRead, a reusable reading control layer that governs content admission without modifying the agent model. Its *Read Intent Protocol* tracks evolving needs and source coverage; *Adaptive Reader Backends* select how evidence is located and exposed; and the *Budgeted Read Gate* bounds cumulative raw exposure per episode with faithful full-view fallback. A unified Reader interface supports interchangeable evidence acquisition across heterogeneous resources. Across six models, including Claude Opus 5, and five scenarios, SparseRead reduces total tokens by up to 92.9% and wall time by up to 89.0%, preserving or improving task quality in 97% settings. On Web deep-search tasks, it reduces tokens by 31.0–43.0% and task completion time by 23.0–37.0%. Ablations validate the three designs, while integration with three agent frameworks demonstrates portability.

CCS Concepts

• **Computing methodologies** → **Intelligent agents**.

Keywords

LLM agents, sparse reading, reading control, evidence acquisition, token efficiency

1 Introduction

Large language model (LLM) agents have extended the scope of language models from single-turn generation to long-horizon task execution [27, 43]. By coupling reasoning with tool use and access to external artifacts, agents support increasingly complex applications across software engineering, scientific analysis, and interactive environments [3, 42, 47]. For Web agents, these artifacts include webpages, online reports, code repositories, and structured responses from Web APIs. This capability, however, introduces a markedly different inference profile: each step adds new observations that may remain in subsequent model calls, causing token consumption, latency, and serving cost to grow rapidly with trajectory length. This raises a question for agent infrastructure: what should an agent read from a Web resource before its next decision?

Reading constitutes a major source of this growth. Prior work reports that external observations account for 60–80% of the tokens processed in agent trajectories [37]. Yet an agent may receive an entire page, file, or tool result when its next decision requires only a few source passages. Paired executions further show that unmodified agents can consume up to 17.3× more tokens without achieving higher task quality. Excess content is not necessarily benign; by diluting relevant evidence, it can induce erroneous tool use, source confusion, and failures to stop. We refer to this mismatch as over-reading: external artifacts are accessed at object granularity, although each decision often requires only a bounded evidence slice. When admitted early, unnecessary content may also be carried into many later model calls.

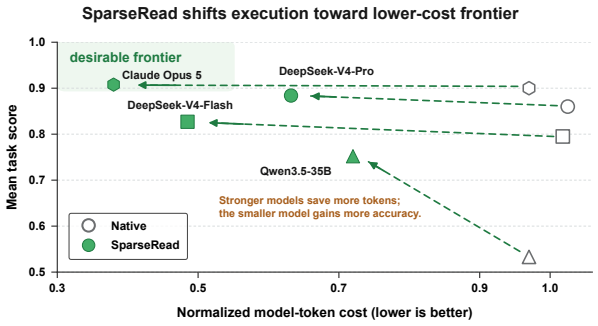


Figure 1: Result teaser. SparseRead shifts model execution toward a lower-cost, quality-preserving frontier.

Existing techniques address parts of this problem without jointly controlling artifact reading. Long-context architectures and KV-cache compression reduce the compute and memory overhead of context, but do not decide which source content the model should see [25, 45]. ACON and observation masking reduce tool observations or their persistence in the trajectory [17, 26]. Retrieval, including iterative and agentic approaches, can acquire passages on demand [1, 12, 41], but does not by itself provide a common artifact-level contract for evolving read intents, explicit source coverage, bounded exposure, and full-view fallback. A general mechanism is still needed to govern artifact access before unnecessary content enters the agent context.

Human reading provides a natural abstraction for this missing capability. Human readers first clarify their purpose, inspect an artifact's structure when needed, focus on relevant passages, and revisit unresolved details. We call this capability Sparse Reading: the agent reads only what its current reasoning requires and expands the read only when additional evidence is needed.

Turning this pattern into an agent mechanism raises two challenges. First, information needs change as an agent works through subgoals and verification steps. Each partial read must clarify its

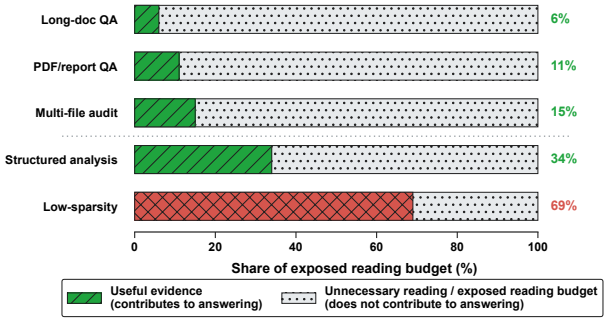


Figure 2: Reading-budget breakdown across representative workloads. Evidence share is the ratio of manually annotated supporting evidence to the source content exposed to the native agent, both measured in characters.

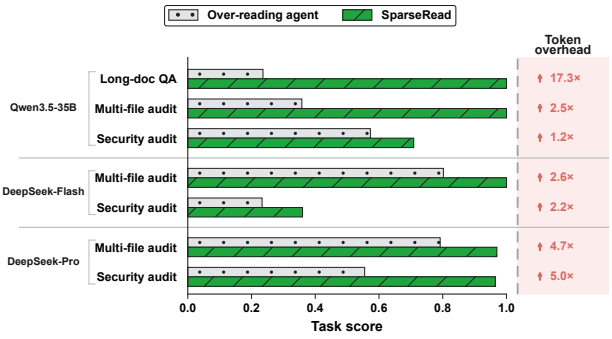


Figure 3: Quality-efficiency comparison. Sparse reading matches or improves task quality while using fewer tokens than full reading. More exposed context does not automatically produce better answers.

purpose, record what has been inspected and what remains uncovered, and use an appropriate way to acquire evidence from the current artifact. Otherwise, the agent may repeatedly revisit the same source. Second, sparse access is not always economical. Small objects and requests that require a complete model-visible view may make repeated partial reads more costly than direct access. These challenges call for explicit evidence state, flexible evidence acquisition, and online control over when to continue reading or fall back to a full view.

We present SparseRead, a framework that controls artifact admission before broad content enters the model context. The agent states its per-access needs through a Read Intent, and the reading layer returns an EvidencePack containing bounded, source-anchored evidence and explicit coverage; the agent decides whether the evidence is sufficient. SparseRead combines a Read Intent Protocol, Adaptive Reader Backends, and a Budgeted Read Gate. SparseRead can be integrated into existing agent frameworks and used with existing LLMs, without changes to model parameters or a separate generative compression model.

This paper makes the following contributions:

- **Read Intent Protocol.** We introduce a cooperative protocol that turns broad artifact access into a bounded and stateful reading process. It makes per-access evidence needs and source coverage explicit, while leaving sufficiency and stopping decisions to the agent.
- **Adaptive Reader Backends.** We separate reading control from evidence acquisition through a unified Reader interface. Reader configurations combine artifact-specific location or processing methods with exposure forms, selected using calibrated evidence reliability and estimated reading costs.
- **Budgeted Read Gate.** We introduce an inference-time controller that regulates when sparse reading is attempted and when a full view is preferable. An episode-level exposure budget bounds cumulative raw artifact exposure while retaining an explicit full-view fallback.
- **Evaluation.** We compare SparseRead with strong native agents and static and agentic RAG across five scenarios, and examine the contribution and effective regime of each component.

2 Quantifying the Cost of Over-Reading

We first quantify the sparsity of annotated supporting evidence, then examine how broad exposure accumulates across later calls. Together, these observations make over-reading a measurable agent execution problem.

2.1 Useful Evidence Is Sparse

Figure 2 shows reading-budget breakdowns across representative workloads. For long-document QA, annotated supporting evidence covers a median of 3.17% of source tokens per question and 2.89% across five questions from one document in LooGLE; the share is about 5.8% for analyzed answerable Qasper questions with evidence annotations. These annotation ratios differ from the figure's exposure shares and do not measure the reading needed to locate and verify evidence. Moreover, different questions may require evidence from different parts of the same document. This motivates evidence acquisition guided by the current information need rather than a fixed summary of the entire source.

This observation also highlights three distinct quantities: processing scope, model exposure, and evidence coverage. Processing scope describes how much source content a Reader examines or computes over, whereas exposure measures what enters the model context. A Reader can scan an entire table to compute a column sum while exposing only the result, row count, and source. Coverage, in turn, records which slots in the current Read Intent have supporting evidence and which remain unresolved, without judging sufficiency. These quantities need not scale together. In particular, reducing model exposure does not require reducing the amount of source content processed, allowing Sparse Reading to benefit both localized retrieval and exact whole-object operations.

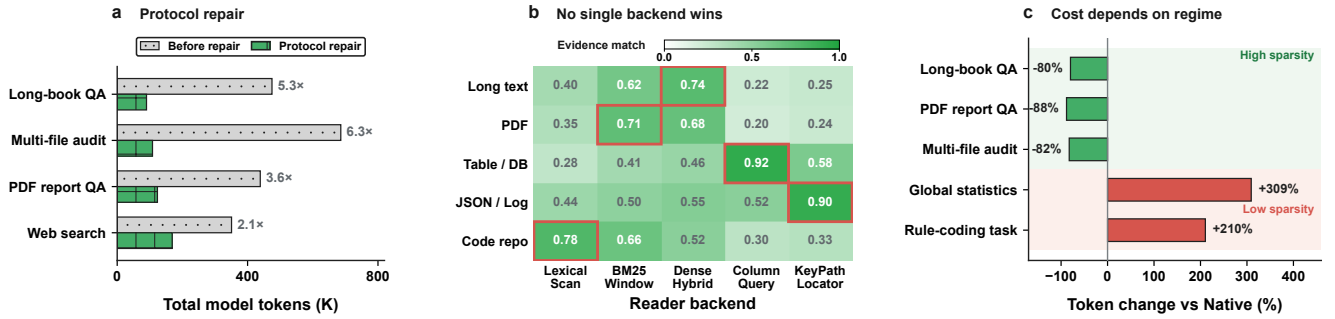


Figure 4: Challenges of controlled reading. (a) Protocol repair reduces total model tokens. (b) The best evidence location method varies across artifact types. (c) Token cost depends on the reading regime.

2.2 Over-Reading Compounds Across Model Calls

The cost of a broad read can extend beyond its initial admission. Content exposed at step i may remain in later model calls. Let e_i denote newly exposed content and R_i the number of subsequent calls retaining it. The resulting replay-weighted exposure is $C_{\text{exposure}} = \sum_i |e_i|R_i$. This measures logical context replay, not billed tokens or physical prefill work; prefix caching can lower their marginal costs without removing retained content.

If broad reading always improved quality, over-reading would be a cost-only problem. Figure 3 shows otherwise. In paired high-sparsity runs, full-read agents spend up to 17.3 times more tokens than SparseRead while failing to improve task score. Across model and task groups, selective reading preserves or improves score while reducing exposed context. The result does not imply that less context is always better; it shows that broad context is not a free substitute for reading control.

These observations show why reading should be controlled, but not what evidence an agent should seek, how its coverage is tracked, or when to switch to a full view.

3 Why Retrieval or Compression Alone Is Not Enough

Problem formulation. At step t, let o denote the current external artifact and h_t the current Read Intent. The core challenge is to choose the content x_t exposed to the model before its next decision, while preserving task-relevant evidence, tracking coverage of h_t, and controlling the cost of subsequent reads.

Retrieval can locate passages, including through iterative queries, while compression can reduce the content admitted to context. Yet neither alone establishes a common reading contract across heterogeneous artifacts. Two challenges remain: tracking evolving evidence needs and deciding when selective reading is no longer economical.

3.1 Challenge 1: Evolving Needs and Uncertain Evidence Coverage

An agent's information needs evolve during execution. The opening request establishes a session-level objective, but subsequent steps introduce subgoals, follow-up questions, and verification needs. Each access therefore requires an intent specific to the current decision. Iterative retrieval can also update queries; the missing abstraction is a consistent intent and evidence record across artifact types and access tools.

Figure 4(a) shows that protocol repair reduces total model tokens by 2.1–6.3× across long-book QA, multi-file audit, PDF report QA, and Web search. Without a record of inspected regions and unresolved evidence slots, an agent may reopen artifacts or continue searching unnecessarily. Coverage records what has been inspected or found, rather than declaring the evidence sufficient. Source-anchored evidence and explicit coverage help the agent decide what to read or verify next, and when to stop.

Evidence acquisition also depends on the artifact. Figure 4(b) shows that the best evidence location method varies across artifact types: dense hybrid retrieval works best for long text, while column queries and key-path lookup are more effective for tables and JSON/logs, respectively. Locating evidence and deciding how much to expose are separate choices, motivating configurable Readers rather than a single retrieval pipeline.

3.2 Challenge 2: The Regime Boundary Is Unknown Before Reading

Sparse Reading is well-suited for tasks that require extracting localized information from large artifacts. Long-document QA, PDF comprehension, and audit bundles exhibit this characteristic. However, some tasks favor a full view: small objects may fit directly into context, and some requests require full model-visible access. For these tasks, sparse reading may add overhead without reducing token usage. In contrast, an exact whole-table operation can process the full source yet expose only a compact, verifiable result.

Figure 4(c) illustrates this boundary. Sparse Reading reduces token consumption by 80–88% on high-sparsity QA and audit tasks, while forced sparse reading increases tokens by 309% for global

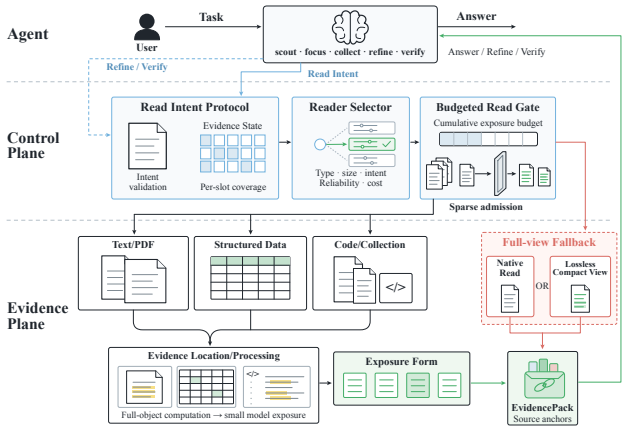


Figure 5: SparseRead separates the agent-facing control plane from replaceable Reader Backends. The Read Gate selects the intervention mode, while the protocol governs evidence acquisition, state updates, and fallback.

statistics and 210% for a rule-coding task, where repeated selective access outweighs its savings. Sparse Reading should therefore have an explicit fallback rather than being applied uniformly.

This boundary cannot always be determined before reading: the evidence needed for a decision and the cost of subsequent refinement remain uncertain. Object size or type alone is insufficient. Rather than predicting the optimal regime in advance, an online controller can attempt sparse reading while limiting cumulative exposure against the cost of a faithful full view, switching before the next batch exceeds the budget.

Design implication. Together, these challenges show that Sparse Reading needs more than retrieval or compression. The Read Intent Protocol tracks evolving needs and evidence coverage, Adaptive Reader Backends choose how evidence is located and exposed, and the Budgeted Read Gate controls exposure and full-view fallback, forming a unified reading loop.

4 SparseRead Design

SparseRead separates reading control from evidence acquisition. The control plane interprets an agent's reading intent, selects among eligible Reader configurations, maintains the evidence record, and governs what enters the model context. The evidence plane contains Readers that declare their supported artifact types, actions, and execution costs. Readers acquire evidence but do not decide when they are invoked. Figure 5 illustrates the architecture. Throughout the design, we distinguish the scope processed by a Reader, the content exposed to the model, and the evidence coverage of the current intent.

The two planes share a common contract: **Read Intent → EvidencePack**. It covers targeted passages, faithful compact views, and results of exact source operations without changing the agent-facing interface. Ordinary sparse reads leave preceding turns intact, allowing exact-match prefix reuse. A subsequent fallback may retire earlier sparse observations to avoid further replay, potentially losing prefix-cache reuse from the edited point onward.

4.1 Read Intent Protocol

SparseRead treats reading as a sequence of evidence requests guided by the agent. At each access, the agent supplies a structured **Read Intent** describing what it needs for its next decision. This access intent takes precedence over a session intent derived from the user's opening request. The session intent fills in unspecified details rather than overriding a new subgoal or follow-up question. An actionable intent proceeds directly to evidence acquisition; an underspecified one receives a bounded **Object Preview** for refinement. The agent can also request the same preview explicitly through *scout*.

Each read returns an **EvidencePack** with source-anchored evidence, the scope examined or processed, the amount exposed, and coverage for individual evidence slots. Coverage identifies which needs have supporting evidence and which remain unresolved. It does not establish whether the evidence is sufficient to answer the task. The agent decides whether to answer, refine, or verify; neither the protocol nor the Reader declares completion.

The protocol also maintains the validity and relevance of prior evidence. A SHA-256 fingerprint identifies the artifact version; when it changes, evidence and coverage recorded for that artifact are invalidated. On an unchanged version, earlier spans remain valid but may not cover a new intent. Lightweight validation checks for identifiable slots and consistent scope requirements using structural signals rather than task-specific rules. Vague intents trigger an Object Preview, while inconsistent scope requests are flagged and resolved according to their stated coverage requirements.

The agent interacts through five stable actions, $\mathcal{A} = \{\text{scout}, \text{focus}, \text{collect}, \text{refine}, \text{verify}\}$. The selected Reader maps each supported action to operations over its artifact type, so adding Readers does not expand the agent's action set. A **reading episode** is a sequence of accesses to one artifact version under a single reading objective. Refinement and verification within an episode share its evidence record and exposure budget, even as the access intent changes. A distinct objective starts a new episode. Algorithm 1 summarizes one access; the agent, not the protocol, decides whether to make another.

4.2 Adaptive Reader Backends

A Reader is defined by the artifacts it handles, not by a particular retrieval algorithm or evaluation scenario. Each Reader registers its supported artifact types, agent actions, and cost profile behind the unified interface. Within a Reader, evidence acquisition has two related choices: **how to locate or process evidence**, and **what to expose to the agent**. For example, a text or PDF Reader may locate passages using lexical matching, BM25 windows, or hybrid retrieval. A structured Reader may select columns or key paths, or execute an exact computation; a collection Reader may gather evidence across files. These are location or processing methods within Readers, not separate Reader types.

The exposure choice is distinct. Depending on the intent, a Reader may return anchored snippets, a lossless compact view of the entire artifact, or the result and provenance of an exact operation. Not every method supports every exposure form. We define a **Reader configuration** r as a supported combination of a registered Reader, its evidence location or processing method, and an exposure form. All configurations produce the same EvidencePack

Algorithm 1 SparseRead Access Protocol

Input: artifact o, access intent h_a (optional), session intent h_s, agent action a; persistent episode state S

$S \leftarrow \text{SyncArtifactVersion}(S, \text{Fingerprint}(o))$
$(h, \text{flags}) \leftarrow \text{Validate}(\text{ResolveIntent}(h_a, h_s))$
$(H, E, f, L) \leftarrow \text{EnsureEpisode}(S, o, h)$ {fixed full view f}
if $\text{NeedsFullModelView}(h)$ **or** $\text{NativeFit}(o, h, L)$ **then**
  return $\text{FullViewFallback}(f, H)$
if $\text{Underspecified}(h)$ **or** $a = \text{scout}$ **then**
  return $\text{ObjectPreview}(o, \text{flags})$ {bounded and reusable}
$r \leftarrow \text{SelectReader}(o, h, a, H)$ {reliability and cost}
if $r = \text{none}$ **then return** $\text{FullViewFallback}(f, H)$
$b \leftarrow r.\text{Prepare}(o, h, a, H)$
if $b = \text{none}$ **or** $\text{EstimatedSize}(b) > L - E$ **then**
  return $\text{FullViewFallback}(f, H)$
$x \leftarrow \text{SerializeIntact}(b, \text{limit} = L - E)$
if $x = \text{overflow}$ **then return** $\text{FullViewFallback}(f, H)$
$H \leftarrow \text{Record}(H, x, h, \text{Fingerprint}(o), \text{Scope}(b), \text{Coverage}(b, h))$
$E \leftarrow E + \text{PayloadSize}(x)$; $\text{SaveEpisode}(S, H, E)$
return $\text{EvidencePack}(x, \text{Scope}(b), \text{Coverage}(H, h), \text{PayloadSize}(x), \text{flags})$

$\text{FullViewFallback}(f, H)$: retire earlier sparse observations; deliver f once (or reuse it if already exposed); close episode.

contract. This separation allows both new artifact types and improved location methods to be introduced without changing the agent's reading interface. The evidence matching differences across artifact types in Section 3 motivate selecting configurations rather than imposing one retrieval pipeline.

SparseRead adapts this choice to the artifact type, size range, and Read Intent. Let g denote this request group and $\mathcal{R}_{o,h}$ the compatible configurations for artifact o and intent h. The reading layer selects

$$r^* = \arg\min_{r \in \mathcal{R}_{o,h}} \widehat{C}(r \mid g) \quad \text{s.t.} \quad \widehat{p}_{\text{valid}}(r \mid g) \geq \tau,$$

where $\widehat{C}$ estimates evidence exposure, Reader execution, and likely follow-up reads under a consistent cost measure. The reliability estimate $\widehat{p}_{\text{valid}}$ is calibrated on annotated development requests using an explicit per-request evidence success criterion. Average evidence recall alone is not a probability of valid evidence. When no configuration meets the reliability requirement, the system falls back to a full view rather than silently relaxing the threshold.

Read outcomes can further inform cost estimates: repeated access or escalation indicates extra reading work, whereas a verification request is not itself a failure. Such behavioral signals do not establish evidence correctness. Likewise, source anchors and fingerprints establish provenance, not relevance or sufficiency. This division keeps reliability calibration separate from cost adaptation. Specialized Readers and new evidence methods continue to use the same interface.

4.3 Budgeted Read Gate

The Read Gate determines when selective access is worthwhile and when to fall back to a complete view. Let $f(o)$ be the least expensive **faithful full view** available for the current request. It may be the original native read or a lossless compact representation of the entire artifact, provided it preserves the access required by the intent. Let $L = |f(o)|$ be its exposed length. Thus, a compact full view offered by a Reader can lower the Gate's budget. The selected $f(o)$ and L remain fixed within an episode; switching to a different full view starts a new episode. Both L and the cumulative sparse payload E use the same unit, character length in our implementation, rather than disk size or replay-weighted tokens.

The Gate follows three rules. First, it uses full-view access when the model must inspect the complete content or no suitable Reader is available. Second, it bypasses sparse reading for small artifacts. Otherwise, it begins with sparse evidence and enforces $E \leq L$. Before delivering a batch, it compares its estimated size with the remaining budget. Serialization then checks the actual payload against the same limit. If the next batch cannot fit intact, it is withheld and the Gate delivers $f(o)$ instead. Evidence is never truncated in a way that breaks its meaning or source anchors.

Full source processing does not necessarily require full model exposure. A table Reader, for example, can scan every row to calculate a sum and expose only the result, row count, and provenance. The Gate admits this operation to the Reader path because the exposed result is small, even though the processing scope is complete. When a full view has been delivered as evidence, subsequent fallback reuses that view rather than exposing it a second time.

Proposition 4.1 (Native-relative exposure bound). *Consider a reading episode on a fixed artifact version with a fixed full view $f(o)$ of length L. If the Gate admits at most L of cumulative unweighted sparse payload and delivers at most one full view on fallback, the episode's total unweighted artifact exposure is at most $2L$, excluding separately bounded preview and protocol metadata.*

The argument is immediate: sparse batches contribute at most L and a full view adds at most L (proof in Appendix A). When an ordinary native full read is among the admissible representations, L is no greater than its length, so the bound is also at most twice the exposure of that native read. The bound limits the downside of selective reading; savings arise when the agent can finish with much less than L evidence. It is not a bound on replay-weighted context, billed tokens, task latency, or performance against an oracle that already knows where the answer lies. On fallback, earlier sparse observations are retired from later prompts to avoid continued replay, although their previous cost cannot be recovered. The guarantee applies per episode, not across an entire agent session.

5 Implementation and Agent Integration

Agent harness integration. The SparseRead core is a shared runtime. It owns the Read Intent Protocol, the Read Gate, the evidence record, the Readers, and the bridge server. Each agent harness adds only the framework-specific surface needed to route broad object access into that runtime. NanoBot registers the SparseRead tools directly. OpenCode and OpenClaw use thin plugin or bridge adapters that expose `sro_preview` and `sro_read`, retain `sro_card` for compatibility, and translate local read and open events into the same core calls. Model-facing guidance such as `SKILL.md` tells the agent when to start from preview, when to issue a targeted read, and when to return to native access. This guidance is advisory: intent validation and the exposure budget are enforced in the core, so no harness depends on the model's compliance for its guarantees.

Readers and the evidence record. The implementation includes Readers for text and PDF, structured data (tables and JSON), multi-file collections, and script libraries. Text and PDF Readers extract page or heading anchors and return bounded local passages. Table

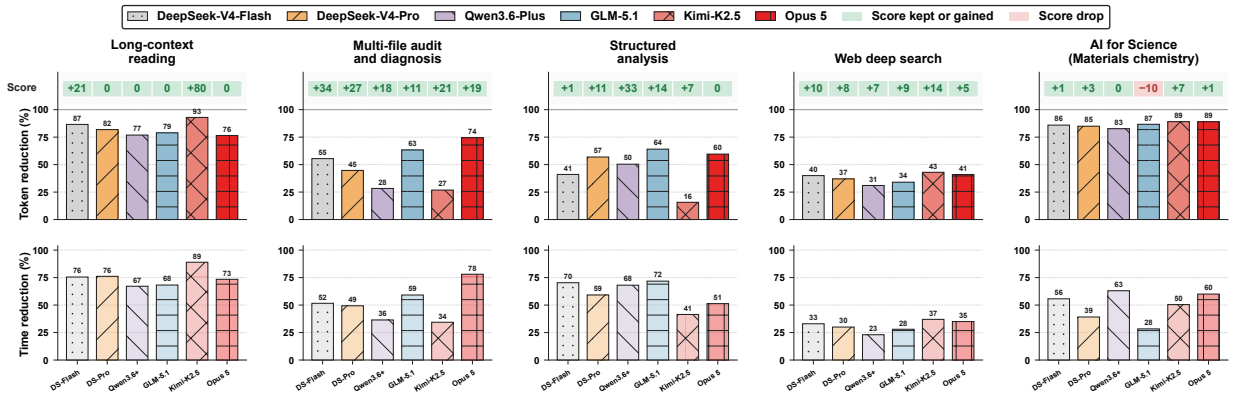


Figure 6: SparseRead results across six models and five scenarios on NanoBot. The upper panels report total-token reduction relative to Naive, with markers showing the corresponding score delta. The lower panels report end-to-end wall-time reduction. Positive bars indicate lower cost.

Readers expose the schema, samples, and row and column projections. They can also run an exact operation over the full table and return only the result with the row count and source provenance, or return calculation-ready TSV fragments when the agent computes itself. Collection Readers build an inventory before selecting candidate files. Each Reader receives the artifact, the Read Intent, and the current evidence record, and returns an EvidencePack with source anchors, the scope processed, and per-slot coverage with unresolved requirements. A Reader declares its artifact types, actions, and cost profile when it registers, so a new Reader becomes available to every harness without adapter changes.

6 Evaluation

6.1 Experimental Setup

The evaluation addresses four research questions. **RQ1** examines whether SparseRead improves reading efficiency across models and scenarios without sacrificing task quality. **RQ2** compares SparseRead with retrieval and context compression in both token cost and end-to-end latency. **RQ3** examines the effects of intent handling, Reader location methods, and Gate budget checks on evidence quality and reading efficiency. **RQ4** evaluates whether the same SparseRead design transfers across agent frameworks.

Tasks and scenarios. We organize the main evaluation around five scenarios, with task sources specified separately for each. *Long-context reading* uses document-comprehension tasks from LooGLE and PinchBench, together with a fact-extraction task adapted from Workspace-Bench-Lite [18, 23, 35]. *Multi-file audit and diagnosis* uses QwenClawBench tasks that require locating and reconciling evidence across workspace files [33]. *Structured analysis* combines QwenClawBench tasks with spreadsheet-manipulation tasks adapted from SpreadsheetBench Verified [30, 33]. *Web deep search* draws tasks from FRAMES and BrowseComp-Plus, requiring iterative search and evidence synthesis across multiple documents [8, 20]. *AI for Science* uses LAMMPS script-generation tasks adapted from MDGYM [21], with scientific script libraries serving as reading resources. Appendix B gives detailed source and adaptation descriptions. Native-fit controls are treated separately in Appendix C, rather than as a main scenario.

Models. The main NanoBot experiment includes six models across three groups. Qwen3.6-Plus and DeepSeek-V4-Flash represent *medium models*. DeepSeek-V4-Pro, GLM-5.1, and Kimi-K2.5 represent *leading large open models*, while Claude Opus 5 represents a *leading commercial closed model*. This selection tests whether sparse reading remains effective across model sizes and whether its benefits persist for leading models with strong reasoning and tool use capabilities.

Agent frameworks. We integrate SparseRead into *NanoBot v0.2.0*, *OpenCode v1.17.14*, and *OpenClaw v2026.6.11*. All integrations share the same reading protocol and Reader Backends, while thin adapters connect the protocol to each framework's tool interface. The cross-framework experiment uses six models and requires no model retraining.

Baselines. We compare SparseRead with Naive and four alternatives. *Naive* preserves the unmodified framework's agent behavior, including autonomous search and partial reads through native tools. *RAG Static* retrieves and reranks evidence once, then generates from a fixed evidence budget without further source access. *RAG Agent* retrieves iteratively with the same retrieval service, retaining native tools under SparseRead's iteration and timeout limits. *ACON* compresses both environment observations and interaction history during agent execution [17]. *Obs. masking* follows the fixed positional policy from *Complexity Trap*, retaining the latest ten observations while masking older ones [26]. Configuration details appear in Appendix D.

Metrics. All costs are measured end to end, including retrieval and reranking for the RAG baselines and auxiliary model calls for ACON. We report total token reduction, task-score change, and wall-time saving relative to Naive; a positive delta indicates an improvement. Figure score strips express score changes in percentage points. Component ablations additionally examine protocol diagnostics, evidence hit rate, and raw artifact exposure.

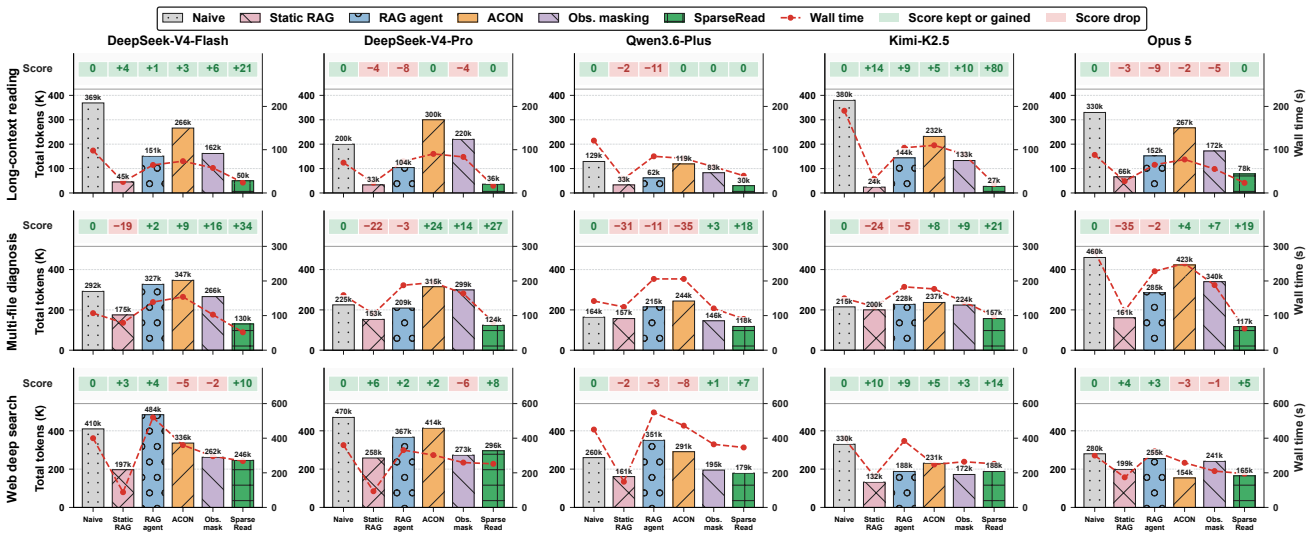


Figure 7: Six methods are compared across three scenarios and five models by total tokens, wall time, and score changes.

Table 1: End-to-end SparseRead results across three agent frameworks and six models.

Framework	Model	Token red.	Score Δ	Time save
NanoBot	Qwen3.6-Plus	69.0%	+0.039	64.4%
	DeepSeek-Flash	22.4%	+0.140	25.6%
	DeepSeek-Pro	71.3%	−0.015	73.1%
	GLM-5.1	61.0%	+0.012	52.6%
	Kimi-K2.5	92.9%	+0.598	85.4%
	Opus 5	58.6%	+0.024	54.2%
OpenCode	Qwen3.6-Plus	25.2%	+0.333	22.8%
	DeepSeek-Flash	71.8%	+0.143	64.9%
	DeepSeek-Pro	58.5%	−0.013	62.9%
	GLM-5.1	75.4%	+0.033	68.9%
	Kimi-K2.5	83.7%	−0.031	73.7%
	Opus 5	46.8%	+0.018	42.7%
OpenClaw	Qwen3.6-Plus	46.7%	+0.032	56.8%
	DeepSeek-Flash	28.7%	+0.062	26.5%
	DeepSeek-Pro	14.5%	+0.151	28.2%
	GLM-5.1	28.7%	+0.118	38.6%
	Kimi-K2.5	42.8%	+0.032	24.6%
	Opus 5	32.5%	+0.011	29.4%

6.2 Effectiveness Across Models and Scenarios

Figure 6 covers 30 settings across six models and five scenarios. SparseRead reduces tokens by 15.7–92.9% and wall time by 23.0–89.0%, with median reductions of 61.4% and 53.7%. Score is preserved or improved in 29 settings; regressions occur for GLM-5.1 on AI for Science.

Long-context tasks save 76.5–92.9% of tokens and 67.1–89.0% of time. Multi-file diagnosis and structured analysis preserve or improve score for every model, with token reductions of 26.8–74.5% and 15.7–64.1%, respectively.

On Web deep search, all six models gain 5–14 percentage points while saving 31.0–43.0% of tokens and 23.0–37.0% of time. AI for Science saves 82.7–89.1% of tokens and 28.3–63.0% of time, with score preserved or improved for five models. Opus 5 saves 41.0–89.0% of tokens and 35.0–78.1% of time across all five scenarios, showing that stronger models still benefit from reading less.

6.3 Comparison with Retrieval and Context-Reduction Baselines

Figure 7 compares six methods in 15 settings across three scenarios and five models. SparseRead preserves or improves score in every setting, saving 27.0–92.9% of tokens and 23.0–88.9% of time. It uses the fewest tokens in six settings and the least time in nine, achieving both minima for all five models on multi-file diagnosis with score gains of 17.8–34.5 percentage points.

RAG Static is faster in all five Web settings but scores below Naive in nine of 15 settings. Under the same iteration and timeout limits, RAG Agent loses score in eight settings and exceeds Naive's time in eight. ACON loses score in five settings and increases both costs in six; observation masking loses score in five. SparseRead's advantage is consistent with controlling admission through explicit evidence needs and coverage, beyond iterative retrieval or compression after acquisition.

6.4 Portability Across Agent Frameworks

Table 1 shows lower token use and wall time in all 18 settings, with score preserved or improved in 15. For NanoBot, OpenCode, and OpenClaw, median token reductions are 65.0%, 65.2%, and 30.6%; time savings are 59.3%, 63.9%, and 28.8%. Three score decreases range from 0.013 to 0.031. Opus 5 saves 32.5–58.6% of tokens and 29.4–54.2% of time, with score gains of 0.011–0.024. Reusing the protocol, Readers, and Gate through thin adapters supports portability without model retraining.

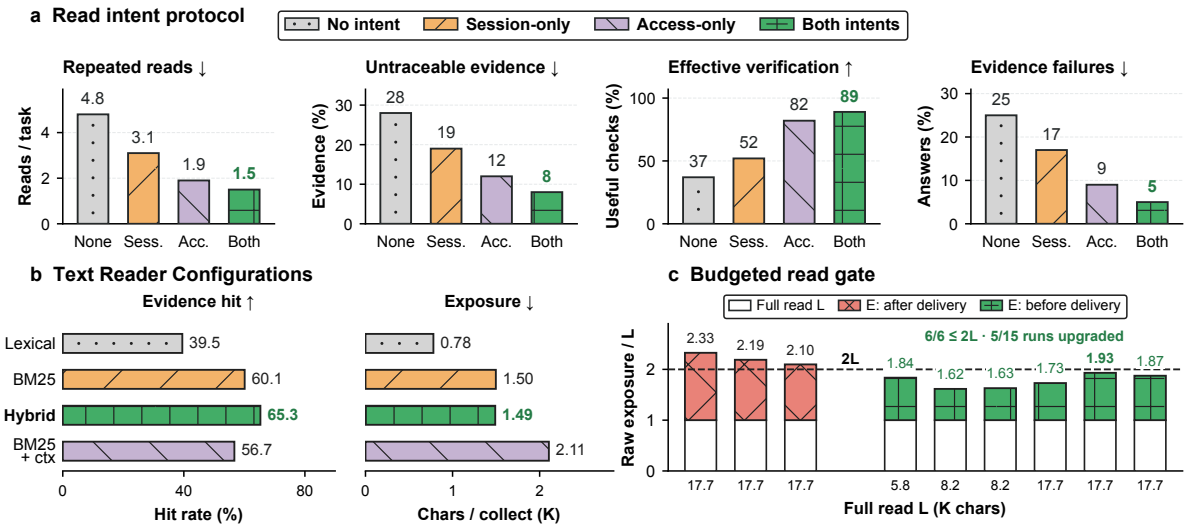


Figure 8: Ablations of intent handling (a), Reader location methods (b), and Gate budget checks (c). Panel (c) compares cumulative raw exposure against the $2L$ bound for checks before and after evidence delivery.

6.5 Ablation of the Three Core Designs

Read Intent Protocol. Figure 8(a) compares no intent, session intent, access intent, and both. Combining intents reduces repeated reads from 4.8 to 1.5 per task (68.8%) and untraceable evidence from 28% to 8%. The share of verification checks that resolve an evidence gap rises from 37% to 89%, while answers failing from missing evidence fall from 25% to 5%. Access intent improves every diagnostic more than session intent alone. Adding session intent further lowers repeated reads from 1.9 to 1.5 and raises effective verification from 82% to 89%, giving the best result on all four diagnostics. This supports combining the broader session objective with the current evidence need to reduce redundant access and improve verification.

Adaptive Reader Backends. Figure 8(b) compares four text Reader location methods on 47 LooGLE documents and 741 questions. A hit requires at least 80% annotated evidence coverage; exposure is median returned characters per collect over five questions. Lexical matching achieves 39.5% hit rate at 780 characters, BM25 60.1% at 1,497, and Hybrid 65.3% at 1,490. Hybrid thus gains 5.2 percentage points over BM25 at essentially the same exposure. Expanding BM25's windows instead increases exposure to 2,106 characters while lowering hit rate to 56.7%. This comparison quantifies location method quality. It does not isolate the incremental benefit of the configuration selector in Section 4.2 over fixed configurations.

Budgeted Read Gate. Figure 8(c) compares checks after and before delivery in separate cohorts, each using five tasks and three repeats on DeepSeek-V4-Flash. Bars normalize cumulative raw exposure by the full view as $(E + L)/L$, using character counts. Checking after delivery allows overshoot: three upgrades reach 2.10–2.33 times the full view. Checking before delivery keeps all six exposure ratios within 1.62–1.93, with upgrades in five of 15 runs.

For tasks unsuited to sparse reading, the Gate bypasses the sparse path and retains native tool use to avoid unnecessary protocol overhead; Appendix C reports the corresponding token-cost and task-score comparison with forced sparse reading.

7 Related Work

Prompt and KV compression. Selective Context, LLMLingua, Gist Tokens, AutoCompressors, and ICAE reduce prompt text or learn compact representations [9, 10, 15, 16, 24, 31, 32]; KV methods optimize cached attention states [5, 13, 25, 28, 29, 39, 46]. SparseRead governs external artifact access and complements these inference optimizations. Ordinary sparse accesses preserve preceding turns, enabling prefix reuse [11].

Agent observations and histories. SWE-Pruner and Squeez prune tool outputs according to task needs [19, 37]; SWE-Pruner Pro selects lines using the agent's internal representations [38]. CoACT, AgentDiet, TACO, ACON, and AgentFold reduce observations or histories [6, 17, 34, 40, 44], while observation masking drops older observations [26]. SparseRead provides a reading control layer above individual compressors: its *pluggable Reader interface* lets SWE-Pruner and Squeez serve as compression backends. The *Read Intent Protocol* coordinates these backends through versioned evidence records linking intent, source coverage, and unresolved needs across refinement and verification.

Active reading and retrieval. ReadAgent combines gist memories with selective lookup [22]; IRCoT and Self-RAG interleave retrieval with reasoning or reflection [1, 36]; Adaptive-RAG routes retrieval by question complexity [14]. Retrieval is one acquisition method under SparseRead's unified evidence contract, alongside structured projections and exact source operations. *Processing scope and model exposure are controlled separately*: a Reader may scan an entire table yet expose only the result and provenance. Its *Budgeted Read Gate* adapts classical ski rental budgeting [4] to cumulative raw exposure across reads, checking each batch before delivery and retaining a faithful full view as fallback within $2L$ per episode.

8 Conclusion

SparseRead provides a reusable reading control layer between agents and external artifacts. Its stateful protocol, pluggable Readers, and Budgeted Read Gate coordinate evolving evidence needs, separate source processing from model exposure, and bound raw exposure while retaining a faithful full view as fallback. The unified interface lets retrieval and compression methods serve as replaceable backends, so advances in evidence acquisition can strengthen the framework across agent implementations. Evaluations show token and latency savings even for leading models, with task quality preserved or improved in most settings; ablations support the complementary roles of the three designs.

References

[1] Akari Asai, Zeqiu Wu, Yizhong Wang, Avi Sil, and Hannaneh Hajishirzi. 2024. Self-rag: Learning to retrieve, generate, and critique through self-reflection. In *International conference on learning representations*, Vol. 2024. 9112–9141.

[2] Beijing Academy of Artificial Intelligence. 2024. BGE-Reranker-v2-M3. Hugging Face model card. https://huggingface.co/BAAI/bge-reranker-v2-m3

[3] Daniil A. Boiko, Robert MacKnight, Ben Kline, and Gabe Gomes. 2023. Autonomous Chemical Research with Large Language Models. *Nature* 624, 7992 (2023), 570–578. doi:10.1038/s41586-023-06792-0

[4] Allan Borodin and Ran El-Yaniv. 1998. *Online Computation and Competitive Analysis*. Cambridge University Press. https://csaws.cs.technion.ac.il/~rani/book.html

[5] Zefan Cai, Yichi Zhang, Bofei Gao, Yuliang Liu, Yucheng Li, Tianyu Liu, Keming Lu, Wayne Xiong, Yue Dong, Junjie Hu, and Wen Xiao. 2024. PyramidKV: Dynamic KV Cache Compression Based on Pyramidal Information Funneling. *arXiv preprint arXiv:2406.02069* (2024). https://arxiv.org/abs/2406.02069

[6] Haorui Chen, Yuancheng Zhu, Yitong Zhang, and Jia Li. 2026. CoACT: Action-Preserving Observation Compression for Coding Agents. *arXiv preprint arXiv:2607.02911* (2026). https://arxiv.org/abs/2607.02911

[7] Jianlv Chen, Shitao Xiao, Peitian Zhang, Kun Luo, Defu Lian, and Zheng Liu. 2024. M3-Embedding: Multi-Linguality, Multi-Functionality, Multi-Granularity Text Embeddings Through Self-Knowledge Distillation. *arXiv preprint arXiv:2402.03216* (2024). https://arxiv.org/abs/2402.03216

[8] Zijian Chen, Xueguang Ma, Shengyao Zhuang, Ping Nie, Kai Zou, Andrew Liu, Joshua Green, Kshama Patel, Ruoxi Meng, Mingyi Su, Sahel Sharifymoghaddam, Yanxi Li, Haoran Hong, Xinyu Shi, Xuye Liu, Nandan Thakur, Crystina Zhang, Luyu Gao, Wenhu Chen, and Jimmy Lin. 2025. BrowseComp-Plus: A More Fair and Transparent Evaluation Benchmark of Deep-Research Agent. *arXiv preprint arXiv:2508.06600* (2025). https://arxiv.org/abs/2508.06600

[9] Alexis Chevalier, Alexander Wettig, Anirudh Ajith, and Danqi Chen. 2023. Adapting Language Models to Compress Contexts. In *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing*. Association for Computational Linguistics, 3829–3846. doi:10.18653/v1/2023.emnlp-main.232

[10] Tao Ge, Jing Hu, Lei Wang, Xun Wang, Si-Qing Chen, and Furu Wei. 2024. In-context Autoencoder for Context Compression in a Large Language Model. In *International Conference on Learning Representations*. https://proceedings.iclr.cc/paper_files/paper/2024/hash/0b276510ec2d3f6613a8b60c41ff0438-Abstract-Conference.html

[11] In Gim, Guojun Chen, Seung-seob Lee, Nikhil Sarda, Anurag Khandelwal, and Lin Zhong. 2024. Prompt Cache: Modular Attention Reuse for Low-Latency Inference. In *Proceedings of Machine Learning and Systems*, Vol. 6. 325–338. https://proceedings.mlsys.org/paper_files/paper/2024/hash/a66caa1703fe34705a4368c3014c1966-Abstract-Conference.html

[12] Kelvin Guu, Kenton Lee, Zora Tung, Panupong Pasupat, and Mingwei Chang. 2020. Retrieval Augmented Language Model Pre-Training. In *Proceedings of the 37th International Conference on Machine Learning (Proceedings of Machine Learning Research, Vol. 119)*. PMLR, 3929–3938. https://proceedings.mlr.press/v119/guu20a.html

[13] Yefei He, Luoming Zhang, Weijia Wu, Jing Liu, Hong Zhou, and Bohan Zhuang. 2024. ZipCache: Accurate and Efficient KV Cache Quantization with Salient Token Identification. In *Advances in Neural Information Processing Systems*, Vol. 37. 68287–68307. https://proceedings.neurips.cc/paper_files/paper/2024/hash/7e57131fdeb815764434b65162c88895-Abstract-Conference.html

[14] Soyeong Jeong, Jinheon Baek, Sukmin Cho, Sung Ju Hwang, and Jong Park. 2024. Adaptive-RAG: Learning to Adapt Retrieval-Augmented Large Language Models through Question Complexity. In *Proceedings of the 2024 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies (Volume 1: Long Papers)*. Association for Computational Linguistics, 7036–7050. doi:10.18653/v1/2024.naacl-long.389

[15] Huiqiang Jiang, Qianhui Wu, Chin-Yew Lin, Yuqing Yang, and Lili Qiu. 2023. LLMLingua: Compressing Prompts for Accelerated Inference of Large Language Models. In *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing*. Association for Computational Linguistics, 13358–13376. doi:10.18653/v1/2023.emnlp-main.825

[16] Huiqiang Jiang, Qianhui Wu, Xufang Luo, Dongsheng Li, Chin-Yew Lin, Yuqing Yang, and Lili Qiu. 2024. LongLLMLingua: Accelerating and Enhancing LLMs in Long Context Scenarios via Prompt Compression. In *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*. Association for Computational Linguistics, 1658–1677. doi:10.18653/v1/2024.acl-long.91

[17] Minki Kang, Wei-Ning Chen, Dongge Han, Huseyin A. Inan, Lukas Wutschitz, Yanzhi Chen, Robert Sim, and Saravan Rajmohan. 2026. ACON: Optimizing Context Compression for Long-horizon LLM Agents. In *International Conference on Machine Learning*. https://arxiv.org/abs/2510.00615

[18] Kilo. 2026. PinchBench: Benchmarking LLM Models as OpenClaw Coding Agents. https://github.com/pinchbench/skill

[19] Ádám Kovács. 2026. Squeez: Task-Conditioned Tool-Output Pruning for Coding Agents. *arXiv preprint arXiv:2604.04979* (2026). https://arxiv.org/abs/2604.04979

[20] Satyapriya Krishna, Kalpesh Krishna, Anhad Mohananey, Steven Schwarcz, Adam Stambler, Shyam Upadhyay, and Manaal Faruqui. 2025. Fact, Fetch, and Reason: A Unified Evaluation of Retrieval-Augmented Generation. In *Proceedings of the 2025 Conference of the Nations of the Americas Chapter of the Association for Computational Linguistics: Human Language Technologies (Volume 1: Long Papers)*. Association for Computational Linguistics, 4745–4759. doi:10.18653/v1/2025.naacl-long.243

[21] Vinay Kumar, Satyendra Rajput, Mausam, and N. M. Anoop Krishnan. 2026. MDGYM: Benchmarking AI Agents on Molecular Simulations. *arXiv preprint arXiv:2605.08941* (2026). https://arxiv.org/abs/2605.08941

[22] Kuang-Huei Lee, Xinyun Chen, Hiroki Furuta, John Canny, and Ian Fischer. 2024. A Human-Inspired Reading Agent with Gist Memory of Very Long Contexts. In *Proceedings of the 41st International Conference on Machine Learning (Proceedings of Machine Learning Research, Vol. 235)*. PMLR, 26396–26415. https://proceedings.mlr.press/v235/lee24c.html

[23] Jiaqi Li, Mengmeng Wang, Zilong Zheng, and Muhan Zhang. 2023. Loogle: Can long-context language models understand long contexts? *arXiv preprint arXiv:2311.04939* (2023).

[24] Yucheng Li, Bo Dong, Frank Guerin, and Chenghua Lin. 2023. Compressing Context to Enhance Inference Efficiency of Large Language Models. In *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing*. Association for Computational Linguistics, 6342–6353. doi:10.18653/v1/2023.emnlp-main.391

[25] Yuhong Li, Yingbing Huang, Bowen Yang, Bharat Venkitesh, Acyr Locatelli, Hanchen Ye, Tianle Cai, Patrick Lewis, and Deming Chen. 2024. SnapKV: LLM Knows What You are Looking for Before Generation. In *Advances in Neural Information Processing Systems*, Vol. 37. 22947–22970. doi:10.52202/079017-0722

[26] Tobias Lindenbauer, Igor Slinko, Ludwig Felder, Egor Bogomolov, and Yaroslav Zharov. 2025. The Complexity Trap: Simple Observation Masking Is as Efficient as LLM Summarization for Agent Context Management. *arXiv preprint arXiv:2508.21433* (2025). https://arxiv.org/abs/2508.21433

[27] Xiao Liu, Hao Yu, Hanchen Zhang, Yifan Xu, Xuanyu Lei, Hanyu Lai, Yu Gu, Hangliang Ding, Kaiwen Men, Kejuan Yang, et al. 2024. Agentbench: Evaluating llms as agents. In *International Conference on Learning Representations*, Vol. 2024. 52989–53046.

[28] Yuhan Liu, Hanchen Li, Yihua Cheng, Siddhant Ray, Yuyang Huang, Qizheng Zhang, Kuntai Du, Jiayi Yao, Shan Lu, Ganesh Ananthanarayanan, Michael Maire, Henry Hoffmann, Ari Holtzman, and Junchen Jiang. 2024. CacheGen: KV Cache Compression and Streaming for Fast Large Language Model Serving. In *Proceedings of the ACM SIGCOMM 2024 Conference*. ACM, 38–56. doi:10.1145/3651890.3672274

[29] Zirui Liu, Jiayi Yuan, Hongye Jin, Shaochen Zhong, Zhaozhuo Xu, Vladimir Braverman, Beidi Chen, and Xia Hu. 2024. KIVI: A Tuning-Free Asymmetric 2bit Quantization for KV Cache. In *Proceedings of the 41st International Conference on Machine Learning (Proceedings of Machine Learning Research, Vol. 235)*. PMLR, 32332–32344. https://proceedings.mlr.press/v235/liu24bz.html

[30] Zeyao Ma, Bohan Zhang, Jing Zhang, Jifan Yu, Xiaokang Zhang, Xiaohan Zhang, Sijia Luo, Xi Wang, and Jie Tang. 2024. SpreadsheetBench: Towards Challenging Real World Spreadsheet Manipulation. *arXiv preprint arXiv:2406.14991* (2024). https://arxiv.org/abs/2406.14991

[31] Jesse Mu, Xiang Lisa Li, and Noah Goodman. 2023. Learning to Compress Prompts with Gist Tokens. In *Advances in Neural Information Processing Systems*, Vol. 36. 19327–19352. https://proceedings.neurips.cc/paper_files/paper/2023/hash/3d77c6dcc7f143aa2154e7f4d5e22d68-Abstract-Conference.html

[32] Zhuoshi Pan, Qianhui Wu, Huiqiang Jiang, Menglin Xia, Xufang Luo, Jue Zhang, Qingwei Lin, Victor Rühle, Yuqing Yang, Chin-Yew Lin, H. Vicky Zhao, Lili Qiu, and Dongmei Zhang. 2024. LLMLingua-2: Data Distillation for Efficient and Faithful Task-Agnostic Prompt Compression. In *Findings of the Association for*

Computational Linguistics: ACL 2024. Association for Computational Linguistics, 963–981. doi:10.18653/v1/2024.findings-acl.57

[33] Qwen Team and Alibaba Data. 2026. QwenClawBench: Real-user-distribution benchmark for OpenClaw agents. https://github.com/SKYLENAGE-AI/QwenClawBench

[34] Jincheng Ren, Siwei Wu, Yizhi Li, Kang Zhu, Shu Xu, Boyu Feng, Ruibin Yuan, Wei Zhang, Riza Batista-Navarro, Jian Yang, and Chenghua Lin. 2026. A Self-Evolving Framework for Efficient Terminal Agents via Observational Context Compression. *arXiv preprint arXiv:2604.19572* (2026). https://arxiv.org/abs/2604.19572

[35] Zirui Tang, Xuanhe Zhou, Yumou Liu, Linchun Li, Yukai Wu, Weizheng Wang, Hongzhang Huang, Wei Zhou, Jun Zhou, Jiachen Song, Shaoli Yu, Jinqi Wang, Zihang Zhou, Hongyi Zhou, Yuting Lv, Jinyang Li, Jiashuo Liu, Ruoyu Chen, Chunwei Liu, GuoLiang Li, Jihua Kang, and Fan Wu. 2026. Workspace-Bench 1.0: Benchmarking AI Agents on Workspace Tasks with Large-Scale File Dependencies. *arXiv preprint arXiv:2605.03596* (2026). https://arxiv.org/abs/2605.03596

[36] Harsh Trivedi, Niranjan Balasubramanian, Tushar Khot, and Ashish Sabharwal. 2023. Interleaving Retrieval with Chain-of-Thought Reasoning for Knowledge-Intensive Multi-Step Questions. In *Proceedings of the 61st Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*. Association for Computational Linguistics, 10014–10037. doi:10.18653/v1/2023.acl-long.557

[37] Yuhang Wang, Yuling Shi, Mo Yang, Rongrui Zhang, Shilin He, Heng Lian, Yuting Chen, Siyu Ye, Kai Cai, and Xiaodong Gu. 2026. SWE-Pruner: Self-Adaptive Context Pruning for Coding Agents. *arXiv preprint arXiv:2601.16746* (2026). https://arxiv.org/abs/2601.16746

[38] Yuhang Wang, Yuling Shi, Shaoqiu Zhang, Jialiang Liang, Shilin He, Siyu Ye, Yuting Chen, Kai Cai, and Xiaodong Gu. 2026. SWE-Pruner Pro: The Coder LLM Already Knows What to Prune. *arXiv preprint arXiv:2607.18213* (2026). https://arxiv.org/abs/2607.18213

[39] Guangxuan Xiao, Yuandong Tian, Beidi Chen, Song Han, and Mike Lewis. 2024. Efficient Streaming Language Models with Attention Sinks. In *International Conference on Learning Representations*. https://proceedings.iclr.cc/paper_files/paper/2024/hash/5e5fd18f863cbe6d8ae392a93fd271c9-Abstract-Conference.html

[40] Yuan-An Xiao, Pengfei Gao, Chao Peng, and Yingfei Xiong. 2026. Reducing Cost of LLM Agents with Trajectory Reduction. *Proceedings of the ACM on Software Engineering* 3, FSE (2026), 1241–1263. doi:10.1145/3797084

[41] Fangyuan Xu, Weijia Shi, and Eunsol Choi. 2023. Recomp: Improving retrieval-augmented lms with compression and selective augmentation. *arXiv preprint arXiv:2310.04408* (2023).

[42] John Yang, Carlos E. Jimenez, Alexander Wettig, Kilian Lieret, Shunyu Yao, Karthik Narasimhan, and Ofir Press. 2024. SWE-agent: Agent-Computer Interfaces Enable Automated Software Engineering. In *Advances in Neural Information Processing Systems*, Vol. 37. 50528–50652. doi:10.52202/079017-1601

[43] Shunyu Yao, Jeffrey Zhao, Dian Yu, Nan Du, Izhak Shafran, Karthik R. Narasimhan, and Yuan Cao. 2023. ReAct: Synergizing Reasoning and Acting in Language Models. In *The Eleventh International Conference on Learning Representations*. OpenReview.net. https://openreview.net/forum?id=WE_vluYUL-X

[44] Rui Ye, Zhongwang Zhang, Kuan Li, Huifeng Yin, Zhengwei Tao, Yida Zhao, Liangcai Su, Liwen Zhang, Zile Qiao, Xinyu Wang, Pengjun Xie, Fei Huang, Siheng Chen, Jingren Zhou, and Yong Jiang. 2025. AgentFold: Long-Horizon Web Agents with Proactive Context Management. *arXiv preprint arXiv:2510.24699* (2025). https://arxiv.org/abs/2510.24699

[45] Manzil Zaheer, Guru Guruganesh, Kumar Avinava Dubey, Joshua Ainslie, Chris Alberti, Santiago Ontanon, Philip Pham, Anirudh Ravula, Qifan Wang, Li Yang, and Amr Ahmed. 2020. Big Bird: Transformers for Longer Sequences. In *Advances in Neural Information Processing Systems*, Vol. 33. Curran Associates, Inc., 17283–17297. https://proceedings.neurips.cc/paper_files/paper/2020/file/c8512d142a2d849725f31a9a7a361ab9-Paper.pdf

[46] Zhenyu Zhang, Ying Sheng, Tianyi Zhou, Tianlong Chen, Lianmin Zheng, Ruisi Cai, Zhao Song, Yuandong Tian, Christopher Ré, Clark Barrett, Zhangyang Wang, and Beidi Chen. 2023. H2O: Heavy-Hitter Oracle for Efficient Generative Inference of Large Language Models. In *Advances in Neural Information Processing Systems*, Vol. 36. 34661–34710. https://proceedings.neurips.cc/paper_files/paper/2023/hash/6ceefa7b15572587b78ecfcebb2827f8-Abstract-Conference.html

[47] Shuyan Zhou, Frank F Xu, Hao Zhu, Xuhui Zhou, Robert Lo, Abishek Sridhar, Xianyi Cheng, Tianyue Ou, Yonatan Bisk, Daniel Fried, et al. 2024. Webarena: A realistic web environment for building autonomous agents. In *International Conference on Learning Representations*, Vol. 2024. 15585–15606.

A Proof of the Native-Relative Exposure Bound

Proof of Proposition 4.1. Fix an episode, so that $f(o)$ and L are fixed. Exposure is measured in the unit of L (characters), and background processing by a Reader is not exposure. As in the proposition, separately bounded preview and protocol metadata are excluded. We first show that $E \le L$ at all times. Initially $E = 0$. The counter changes only at line 16 of Algorithm 1, after line 13 has serialized the batch under the limit $L - E$. The updated value $E + \text{PayloadSize}(x)$ is therefore at most L. Every other branch, including each fallback, leaves E unchanged. Now count what the episode exposes. Admitted sparse evidence accounts for exactly $E \le L$. The fallback delivers $f(o)$ at most once and closes the episode. If $f(o)$ was already exposed as admitted evidence, the fallback adds nothing and the total is $E \le L$. Otherwise it adds L, and the total is at most $E + L \le 2L$. An episode that ends without a fallback exposes $E \le L$. In every case the total is at most $2L$. □

The invariant depends only on the check of the serialized payload (lines 13 and 14). The size estimate at line 11 can only trigger a fallback earlier. Delivering a batch that crosses the budget and escalating afterwards would instead give $2L + \delta$, where δ is the largest batch size. When an ordinary native read is an admissible full view, L is at most its length N, so the exposure is at most $2N$. The constant 2 is tight for this rule: admitted batches totaling just under L, followed by a withheld batch and a fallback, expose just under $2L$. The count is cumulative and unweighted. Retiring earlier observations on fallback reduces future replay but does not refund exposure already paid. The bound therefore says nothing about replay-weighted context, billed tokens, latency, or sessions with several episodes.

B Task Sources and Scenario Construction

Coverage and inclusion. For each scenario, we include *all relevant tasks* from every benchmark listed below, giving *100% coverage of its scenario-relevant task set*. Relevance is determined by the input artifacts and required operations described below, rather than by task scores or observed reading savings; tasks are not sampled or chosen as favorable examples. The coverage denominator is the relevant task set within each benchmark, not unrelated task families in the complete benchmark. Adaptations map the scenario-relevant requirements, source artifacts, and outputs to the agent interface and checker. Multiple questions or follow-up variants over the same source remain related task formulations, rather than independent source documents.

Long-context reading. We include all long-document comprehension tasks from LooGLE [23], all document-comprehension tasks from PinchBench [18], and all annual-report fact-extraction tasks relevant to this scenario from Workspace-Bench-Lite [35]: 100% of the relevant tasks in each source. LooGLE includes single-question, multi-question, and follow-up formulations. Workspace-Bench-Lite tasks are adapted to extracting requested facts, such as research-and-development figures, from company annual reports; they do not reproduce unrelated operations from the complete workspace benchmark.

Multi-file audit and diagnosis. We include all QwenClawBench tasks requiring audit or diagnosis across workspace files [33], covering 100% of its tasks relevant to this scenario. These comprise data-integrity checks, configuration and scheduled-job failures, command-prefix security analysis, and service-audit diagnosis. Each requires locating and reconciling evidence across files, rather than merely retrieving an isolated fact.

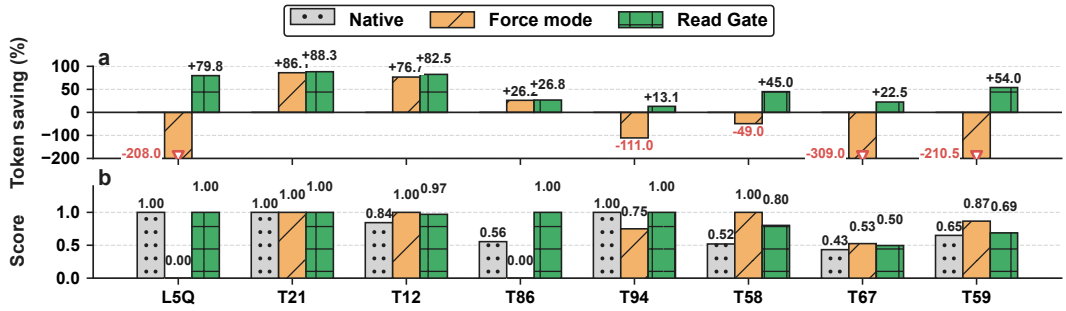


Figure 9: Read-Gate behavior across 8 matched task on Deepseek-v4 Pro. Gray bars denote Native, and colored bars denote SparseRead under the gate-selected policy. Annotations report score changes or cost reductions relative to Naive.

Structured analysis. We include all relevant statistical and financial analysis tasks from QwenClawBench and all spreadsheet-manipulation tasks from SpreadsheetBench Verified that operate on the structured artifacts in this scenario [30, 33], covering 100% of the relevant tasks in each source. Spreadsheet tasks, including lookup and weekday-row correction, are adapted to the agent's execution interface and checker. Coverage refers to these structured tasks, rather than unrelated capabilities in either benchmark.

Web deep search. We include all relevant multi-source search and reasoning tasks from FRAMES and BrowseComp-Plus, covering 100% of the relevant tasks in each benchmark. FRAMES supplies multi-hop questions requiring evidence from multiple sources [20]; BrowseComp-Plus supplies deep-research questions with human-verified supporting documents and challenging negatives in a fixed corpus [8]. Together they evaluate iterative search and selective reading across documents. BrowseComp-Plus uses its fixed corpus rather than unrestricted live-Web browsing.

AI for Science. We include all tasks in MDGYM's LAMMPS portion relevant to script generation, covering 100% of this relevant task set [21]; GROMACS tasks fall outside the scenario. Scientific script libraries provide the external reading resources. The adaptation evaluates script generation and reading behavior, rather than reproducing MDGYM's complete simulation-execution and physical-validation evaluation.

C Native-Fit Controls and Gate Economics

Native-fit controls are drawn from QwenClawBench [33] and include finding the largest file in a directory, implementing a user-discount calculator, and writing a SPARQL query. They probe settings in which direct tool use or a short native read can already be appropriate. They are excluded from the five main scenarios. In the original five-scenario matrix, including these controls, SparseRead reduces token volume and wall time in all 30 model–scenario cells and preserves or improves score in 26. The corresponding results for the four retained scenarios, excluding Web deep search, preserve or improve score in 22 of 24 cells.

Economic boundary of selective reading. The Gate checks whether sparse access is worthwhile before invoking a Reader. When native tools or a short native read already suit the request, it retains that path instead of forcing preview, retrieval, or refinement. This bypass aims to avoid the protocol tax of sparse reading; it does not require the native agent to read every file in full.

Figure 9 reports the supplementary eight-task NanoBot comparison on DeepSeek-V4-Flash, using Naive, forced sparse reading without admission gating, and the earlier Gate policy. The tasks are LooGLE five-question QA (L5Q), PDF comprehension (T21), stock-system audit (T12), command security (T86), exam-system audit (T94), DiD regression (T58), SPARQL generation (T67), and discount calculation (T59). The last three probe the boundary where native processing can be preferable. Forced sparse reading increases tokens by 49.0%, 309.0%, and 210.5% on T58, T67, and T59; the Gate instead saves 45.0%, 22.5%, and 54.0%, with absolute scores of 0.80, 0.50, and 0.69 versus Naive's 0.52, 0.43, and 0.65. It also retains token savings of 79.8%, 88.3%, and 82.5% on L5Q, T21, and T12. Thus selective admission avoids the large overhead of forced sparse reading on these controls while retaining its benefits on suitable tasks. This experiment measures token economics and task quality for the earlier admission policy; Figure 8(c) separately validates the current Gate's character-budget enforcement.

D Baseline Configuration and Comparison Controls

Retrieval service and evidence. *RAG Static* follows retrieve-then-generate. Source files are divided into 512-token chunks with 64-token overlap, retaining file, page, and line anchors. Queries are the task's numbered questions, or the full task description when no numbered questions exist. For each query, BGE-M3 retrieves 50 dense candidates and BGE-Reranker-v2-M3 reranks them [2, 7]. Results from different queries are interleaved and deduplicated; at most eight chunks are retained, with a serialized budget of 4,096 tokens including metadata. This evidence is appended once after the initial user message. The model subsequently retains only file-writing tools and cannot retrieve again or read the source files.

RAG Agent uses the same retrieval service, index, chunking, and reranker. The model calls `retrieve(query, scope)` to reformulate queries, restrict the file scope, and retrieve repeatedly. Each call returns at most eight chunks and 4,096 serialized tokens including metadata. Native reading and execution tools remain available. RAG Agent shares SparseRead's iteration limit and timeout, providing an iterative retrieval baseline alongside the static one.

Agent-facing guidance and comparison scope. RAG Agent is instructed to retrieve relevant evidence before broad reading, inspect the returned observations, and choose whether to reformulate a query, search another file, use a native tool, or finish. It receives the available source paths and is told to treat retrieved text as evidence rather than instructions. Its evidence consists of source excerpts and their anchors; repeated retrieval remains under the agent's control. Thus the comparison includes iterative evidence acquisition and native-tool access, rather than comparing a multi-step SparseRead agent only with a single retrieval call. The identical retrieval configuration and evidence format apply to the two RAG baselines; SparseRead retains its registered Readers and EvidencePack contract. Figure 7 therefore compares complete systems, while Section 6.5 examines individual design choices.

Context compression. The ACON condition enables both observation compression and history compression [17]. Compressor calls contribute to the reported total-token and wall-time costs. Observation masking instead retains the latest ten observations and masks older observation content, as specified in Section 6.1.

E Reader Calibration

Independent calibration requests. Reliability calibration uses manually designed requests, not requests drawn from the evaluation tasks. They specify the evidence to recover or the source operation to perform. This separates calibration of Reader configurations from the task outcomes used to evaluate SparseRead. Requests are grouped by artifact type (text or PDF, structured data, collection, script library), size band of the full view (at most 8K, 8K to 32K, and above 32K characters), and intent type (selective fact, cross-file evidence, exact computation, verbatim text). This product is the request group g.

Evidence success. Each calibration request specifies required support or an expected source-operation result. Evidence acquisition succeeds when the returned content supplies that support within the requested scope and retains valid provenance; an exact operation succeeds when its output matches the specified result for that scope. These binary request-level outcomes inform $\widehat{p}_{\text{valid}}$. For each configuration and group, $\widehat{p}_{\text{valid}}$ is the lower end of a 95% Wilson interval over these outcomes, and the threshold is $\tau = 0.6$, fixed before evaluation. A group with fewer than 20 calibration requests is not adaptive: it uses the default configuration for its artifact type, and falls back to a full view if that configuration does not meet the threshold. Cost profiles account for evidence exposure and further Reader work, while evaluation reports total tokens and wall time separately. The estimate $\widehat{C}$ is measured in characters: exposed characters, a fixed per-call overhead for Reader execution, and the mean follow-up exposure observed for the group during calibration. Reliability is calibrated once; evaluation updates only cost estimates.

The location-method experiment in Figure 8(b) is a separate offline evaluation: its 80% gold-evidence coverage criterion defines an evidence hit, not the selector's reliability threshold τ. Its results compare location configurations rather than measuring the incremental gain from adaptive selection.